Założenia aplikacji

- Interfejs użytkownika zapisany za pomocą języka znaczników wspieranego w danym środowisku (np. XAML, XML)
- Zastosowany liniowy typ rozkładu z kolorem tła AliceBlue (#F0F8FF). Rozkład zawiera:
  - Napis o treści „Notatki”, rozmiar czcionki większy od pozostałych napisów, kolor tekstu MediumBlue (#0000CD)
  - Lista (widok listy), której źródłem jest kolekcja przechowująca notatki jako elementy (kolekcja musi być typu, który jest możliwy do zastosowania z widokiem listy. Dodanie elementów do kolekcji jest widoczne w widoku listy)
  - Napis o treści „Dodanie notatki”, rozmiar czcionki większy od domyślnego rozmiaru, kolor tekstu MediumBlue (#0000CD)
  - Napis o treści „Tytuł notatki: ”
  - Pole do wprowadzania tekstu
  - Napis o treści „Treść notatki: ”
  - Pole do wprowadzania tekstu
  - Przycisk o treści „DODAJ” o kolorze czcionki MediumBlue (#0000CD)

Stan początkowy:

- Wyświetlona jest jedna notatka z dowolnym tytułem i treścią (ilustracja 2 lub 4)

Działanie aplikacji po wybraniu przycisku (ilustracja 3 lub 5):

- Tworzony jest nowy element notatki z wartościami tytułu i treści pobranymi z pól edycyjnych, notatka nie jest ważna
- Element jest dodawany do kolekcji oraz jest widoczny w widoku listy

Kod aplikacji powinien być zapisany czytelnie, z zachowaniem zasad czystego formatowania, należy stosować znaczące nazewnictwo, polskie albo angielskie.

Podejmij próbę kompilacji i emulacji aplikacji. Informacje dotyczące dokumentacji i zrzutów ekranowych umieszczono w części III zadania.

Kod aplikacji przygotuj do nagrania na płytę. W folderze *mobilna* należy umieścić archiwum całego folderu projektu oraz skopiowane pliki z kodem źródłowym, które były modyfikowane.

Część III. Dokumentacja aplikacji

Wykonaj dokumentację do aplikacji utworzonych na egzaminie.

W kodzie źródłowym aplikacji konsolowej za pomocą komentarza utwórz jeden nagłówek do metody według wzoru z listingu 1. Komentarz powinien znaleźć się nad lub pod nazwą funkcji. W miejscu nawiasów <> należy podać odpowiednie opisy. W miejscu autor należy podać numer zdającego.

UWAGA: Dokumentację metody należy umieścić w komentarzu (wieloliniowym lub kilku jednoliniowych). Znajdujący się w listingu 1 wzór dokumentacji jest bez znaków początku i końca komentarza, gdyż te są różne dla różnych języków programowania.

Arkusz pobrany z zawodowe.edu.pl

Listing 1. Wzór dokumentacji metody

```
nazwa:         <nazwa metody>
opis:          <opis metody>
parametry:     <opis parametru1, lub „brak”>
               <opis parametru2>
zwracany typ: <nazwa typu i opis co jest zwracane lub „brak”>
autor:         <numer zdającego>
```

Wykonaj zrzuty ekranu dokumentujące uruchomienie aplikacji utworzonych podczas egzaminu. Zrzuty powinny obejmować cały obszar ekranu monitora z widocznym paskiem zadań. Jeżeli aplikacja uruchamia się, na zrzucie należy umieścić okno z wynikiem działania programu oraz otwarte środowisko programistyczne z projektem lub okno terminala z kompilacją projektu. Jeżeli aplikacja nie uruchamia się z powodu błędów kompilacji, należy na zrzucie umieścić okno ze spisem błędów i widocznym otwartym środowiskiem programistycznym. Wykonać należy tyle zrzutów, ile interakcji podejmuje aplikacja. Wymagane zrzuty ekranu:

- Aplikacja konsolowa – dowolna liczba zrzutów nazwanych *konsola1, konsola2* ...
- Aplikacja mobilna – dowolna liczba zrzutów nazwanych *mobilna1*, *mobilna2* ... (np. stan początkowy, po dodaniu elementu)

Utwórz plik tekstowy i nazwij go *egzamin*. Dokument powinien zawierać informacje o narzędziach wykorzystanych na egzaminie:

- Nazwę systemu operacyjnego
- Nazwy środowisk programistycznych
- Nazwę emulatora dla aplikacji mobilnej
- Nazwy języków programowania

Zrzuty ekranu i dokument umieść w folderze *dokumentacja*.

UWAGA: Nagraj płytę z rezultatami pracy. W folderze z numerem zdającego powinny się znajdować trzy foldery: dokumentacja, konsolowa i mobilna.
W folderze dokumentacja: pliki ze zrzutami oraz plik egzamin.
W folderze konsolowa: spakowany cały projekt aplikacji konsolowej, modyfikowane pliki źródłowe, plik wykonywalny, jeśli istnieje.
W folderze mobilna: spakowany cały projekt aplikacji mobilnej, modyfikowane pliki źródłowe.
Opisz nagraną płytę numerem zdającego i zapakuj w pudełko. Pozostaw ją na stanowisku wraz z arkuszem egzaminacyjnym.

Czas przeznaczony na wykonanie zadania wynosi 180 minut.

Ocenie podlegać będą 4 rezultaty:

- implementacja, kompilacja, uruchomienie programu,
- aplikacja konsolowa,
- aplikacja mobilna,
- dokumentacja aplikacji.

zawodowe.edu.pl - arkusze CKE i pytania egzaminacyjne

Automatyczne odświeżanie widoku listy dla Xamarin z https://docs.microsoft.com

If you want the ListView to automatically update as items are added, removed and changed in the underlying list, you'll need to use an ObservableCollection. ObservableCollection is defined in System.Collections.ObjectModel and is just like List, except that it can notify ListView of any changes:

```
ObservableCollection<Type> CollectionName = new ObservableCollection<Type>();
// add elements to collection
// assign collection to ListView ItemsSource property

//to add element to ListView just add it to collection
CollectionName.Add(collectionElement);
```

Zastosowanie adaptera do odświeżania widoku listy w Android Studio z *https://developer.android.com*

In order to display items in the list, call `setAdapter(android.widget.ListAdapter)` to associate an adapter with the list.

`ArrayAdapter<T>` implements `ListAdapter`, returns a view for each object in a collection of data objects you provide. By default, the array adapter creates a view by calling `Object#toString()` on each data object in the collection you provide, and places the result in a `TextView`. You may also customize what type of view is used for the data object in the collection. To customize what type of view is used for the data object, override `getView(int, android.view.View, android.view.ViewGroup)` and inflate a view resource.

```
public ArrayAdapter (Context context,int resource, T[] objects)
```
You can use embedded resources like `android.R.layout.simple_list_item_1`

```
notifyDataSetChanged();
```
Notifies the attached observers that the underlying data has been changed and any View reflecting the data set should refresh itself.

Example for class Notatka:

```
notes.add(new Notatka("...", "...", true));
ListView notesList = findViewById(R.id.notes);
ArrayAdapter<Notatka> arrayAdapter =
                                new ArrayAdapter<>(this,
                                android.R.layout.simple_list_item_1, notes);
notesList.setAdapter(arrayAdapter);
...
//in AddElement Button listener
notes.add(new Notatka(...));
arrayAdapter.notifyDataSetChanged();
```


Narzędzia dla nauczycieli zawodowych: zawodowe.edu.pl